

Runtime Bypass of Illegal Memory Accesses

CS4110 — Project 1: End-Term Report

Shubham Yadav (112201032)

Anup Kumar (112201042)

Project Mentor :

Dr. Unnikrishnan C

Department of Computer Science
Indian Institute of Technology Palakkad

Submission Date: November 23, 2025

Goal:

Implement a compiler pass on intermediate representation of the program that detects illegal memory accesses and modifies the control flow to skip those instructions instead of aborting.

Problem Statement:

- Memory -unsafe languages like C and C++ are prone to bugs that lead to crash, security holes and undefined behavior.
- Unpredictable software crashes that are difficult to reproduce and debug due to failure of any one component lead crash of complete software.
- Pinpointing the exact location of a memory bug saves countless hours of debugging. Our tool aims to provide precise, actionable logs and availability of the software system even on memory bugs.
- Many critical vulnerabilities such as buffer overflows, use after free , malicious code injection are memory safety problems.
- The implementation is done by using LLVM framework on a code instrumented using Address Sanitizer (ASan).

1. Introduction:

Inspired by **ASan** tool is a fast compiler-based tool for detecting memory bugs in native code and if found halt the program execution.

ASan detects **Stack and heap buffer overflow/underflow, Heap use after free, Stack use outside scope, Double free/wild free**. Average slowdown of the ASan's instrumented program is ~2x .

We are going to build a tool which uses checks inserted by ASan to detect above mentioned memory bugs and instead of aborting the program execution , we are going to log this error and run our own logic to handle the error and continue the program execution.

We try to accomplish this project by using the **LLVM** tool. This work not only aims to build a practical tool for improving software reliability but also serves as a deep dive into the internal workings of modern compilers.

2. ASan:

AddressSanitizer (ASan) is a fast compiler-based tool for detecting memory bugs in native code. ASan detects:

- Stack and heap buffer overflow/underflow
- Heap use after free
- Stack use outside scope
- Double free/wild free

Under the hood, the program's memory accesses will be instrumented with ASan checks and the program will be linked with ASan runtime libraries. As a result, when a memory error is detected, the program will stop and provide information that may help in diagnosing the cause of memory corruption.

2.1 How does it work:

ASan is built upon two key concepts: **shadow memory** and **redzones**. Shadow memory is a dedicated memory region that stores metadata about the application's memory. Redzones are special memory regions placed in between objects in memory. Each byte in shadow memory describes the accessibility status of a corresponding memory chunk ie those memory chunks, typically referred to as "granules," are commonly 8 bytes in size and are aligned to their size

The ASan instrumentation adds checks to the program code to validate legality of the program's memory accesses. By updating the state of shadow memory during the process execution, ASan can verify the validity of memory accesses by checking the granule's value (and so its accessibility status). If a memory granule is fully accessible, a corresponding shadow byte is set to zero. Conversely, if the whole granule is poisoned, the value is negative. If the granule is partially addressable—i.e., only the first N bytes may be accessed and the rest shouldn't—then the number N of addressable bytes is stored in the shadow memory.

The shadow memory mapping and encoding of values (the fact that granules are of 8 bytes in size) allow ASan to efficiently detect memory access errors and provide valuable insight into the problems encountered.

3. Low Level Virtual Machine (LLVM):

LLVM, a compiler framework designed to support transparent, life-long program analysis and transformation for arbitrary programs, by providing high-level information to compiler transformations at compile-time, link-time, run-time, and in idle time between runs.

It is not just a compiler; it is a set of reusable libraries and tools that work together to provide everything from frontend language parsing to backend code generation. LLVM supports not only compilers

but also a broad array of tools such as debuggers, linkers, static analyzers, and more.

The key idea behind LLVM is to separate the **front-end**, **optimizer**, and **back-end** stages of compilation into reusable components.

It allows developers to create custom language frontends, apply sophisticated optimizations, and generate highly optimized machine code for a variety of target platforms.

3.1 LLVM Architecture:

LLVM can be described as a multi-stage, modular system which take high level programs as input and generates highly optimized code for target ISA. It is basically divided into three part:

3.1.1 Frontend:

The frontend component typically includes a lexer, parser, and semantic analyzer that converts the high-level source code into an abstract syntax tree (AST) and then into LLVM IR.

This is the stage where language-specific features and syntax are processed.

Example: Clang is the most common frontend for C, C++ and others like Swift, Rust, Julia, and many more.

3.1.2 Optimization:

Once the source code is converted into LLVM IR, it undergoes a series of optimization passes in the middle-end.

The middle-end performs various types of analysis and transformations, including loop optimizations, inlining, dead code elimination, constant folding, and more.

LLVM provides a command-line tool called **opt** to run, test, and debug these optimization and analysis passes directly on LLVM IR files

For example:

```
opt -passes="pass1,pass2, ." inp.ll -S -o out.ll
```

3.1.3 Backend:

The backend is responsible for generating machine code specific to the target architecture. The LLVM backend is responsible for converting the optimized LLVM IR into assembly code, which is then assembled into binary machine code that can be executed by the processor.

LLVM supports a variety of target architectures, including x86, ARM, PowerPC, MIPS, RISC-V, and more.

3.2 The Heart of LLVM: Intermediate Representation (IR):

It serves as the bridge between high-level programming languages and the machine code that is ultimately executed by hardware. Unlike assembly language or high-level languages like C, C++, Rust, Swift, and others, LLVM IR is designed to be both low-level enough for optimization and platform - independent, making it an ideal representation for cross-platform code generation.

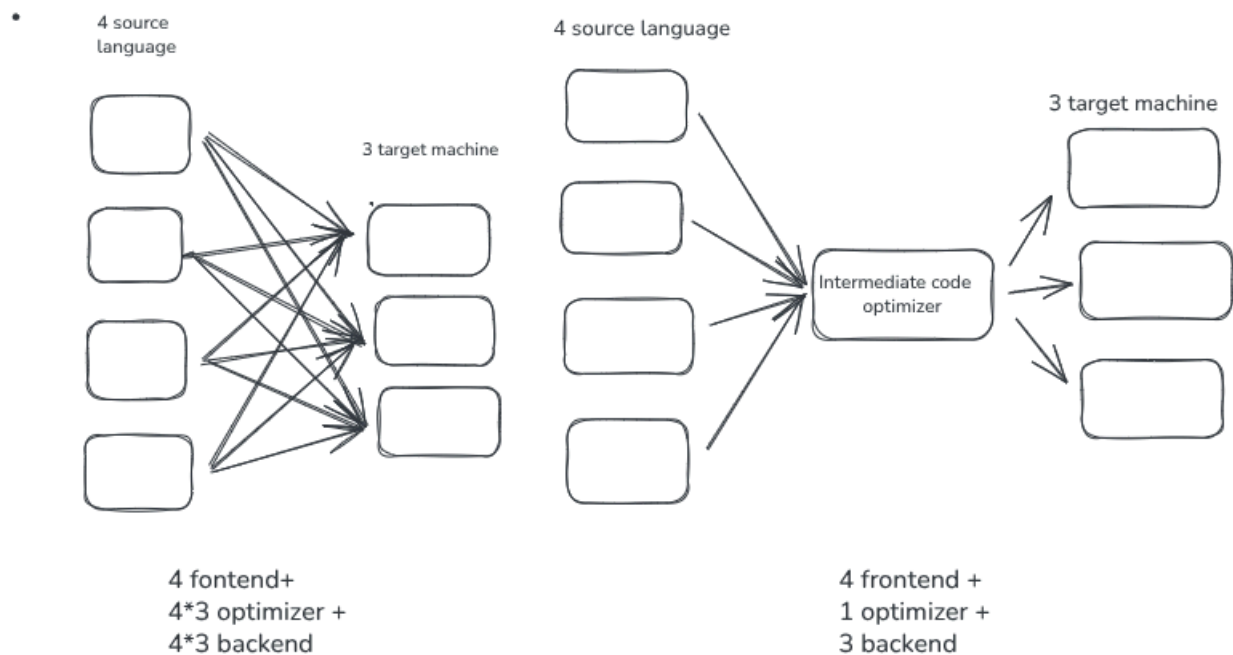


figure 1

3.2.1 Why Intermediate Code? (M x N Problem):

While generating machine code directly from source code is possible, it entails two problems. With m languages and n target machines, we need to write m front ends, $m \times n$ optimizers, and $m \times n$ code generators. The code optimizer which is one of the largest and very-difficult-to-write components of a compiler, cannot be reused.

By converting source code to an intermediate code, a machine-independent code optimizer may be written. This means just m front ends, n code generators and 1 optimizer.

4. LLVM Passes:

Passes perform the transformations and optimizations that make up the compiler, they build the analysis results that are used by these transformations, and they are, above all, a structuring technique for compiler code.

- **ModulePass** – operates on the whole program.
- **FunctionPass** – processes one function at a time.
- **LoopPass / BasicBlockPass** – focuses on specific control-flow regions.

Passes can perform:

- **Analysis**: computing properties (like control flow graphs, dominators, etc.).
- **Transformation**: modifying the IR (e.g., inserting instructions, optimizing code).

5. The ``usacLang`` Recovery Framework: Architecture and Pass Design

5.1 Introduction

We have implemented six LLVM transformation passes: **InitPtrPass**, **AsanPass**, **GEPPass**, **MemcpymvPass**, **FreePass**, and **ReallocPass**.

The first five passes extend the Clang compiler with additional ASan-based instrumentation to form our custom compiler, **usacLang**.

The sixth pass (**ReallocPass**) is independent of ASan and implements its own correctness checks.

Across the second to fifth passes, the core methodology is identical:

we detect potentially faulty instructions using ASan's internal checks (and additional checks inserted by us), and when an instruction is found unsafe, we **reroute the control flow** to the *next safe instruction* using block splitting and CFG transformations. Before rerouting, we **log** the faulty instruction's source location to the same name log file in the same directory.

Before describing each individual pass, we first explain the **common core mechanisms** they rely on.

5.2 Core Methods:

As explained above ASan Insert checks over many instructions that can cause program crashes. With the help of ASan checks and checks we inserted we detect that instruction whose execution can crash the program . And at that point we jump to the log block we log the line where faulty instructions are present in the source file and jump to the next safe instruction and if the check reports safe, normal execution continues.

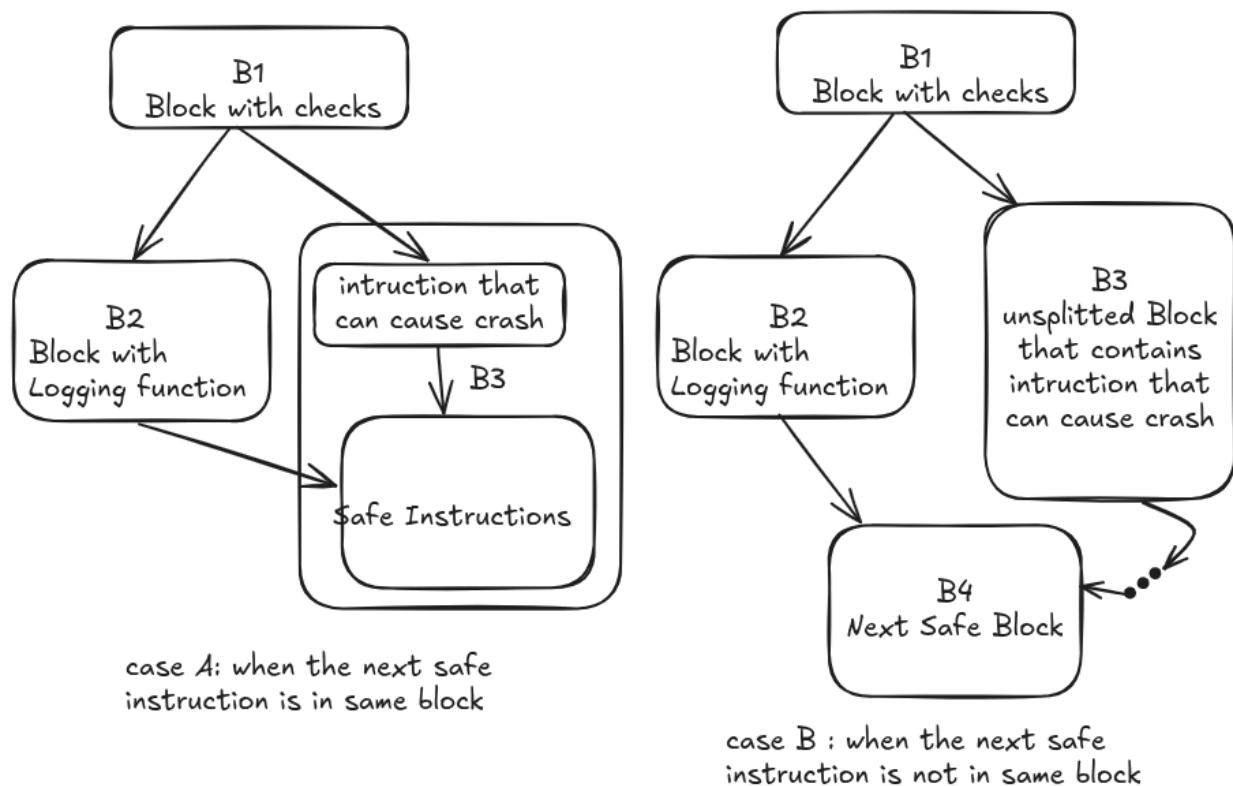


figure 2

Now the problem is how to find the next safe instruction. That safe instruction can be in the same block (block with faulty instruction) figure 2 case A or in different blocks figure 2 case B.

1. NextSafeBlock:

So to find next safe instruction we use the LLVM api `getDebugLoc().getLine()`. We observe that for each IR instruction which is generated corresponding to faulty statements in source code this api returns the same line number equal to line number in source file. So by using this we are finding first instruction whose line number is different in the same block if we found such line we are splitting on that line (that line will go in safe block and rest above are in block with faulty instruction) this is the **case A of figure 2** and if we are unable to find safe instruction in same block then we are applying BFS over Control Flow Graph (CFG) from the block with faulty instruction as source to find the next safe block, which does not contain any instruction related to faulty instruction and that should not be logging block, since it is bfs the next safe block can be at different level from source block, this is the **case B of figure 2**.

There are cases in which both can fail and one such case we found is loop with illegal memory access as shown in figure 3.

// c code snippet

```
1) int* p = NULL;
2) int i = 10;
3) while(i) {
4)   i = i - *p;
   }
```

// corresponding Pseudo IR

```
1) alloc and store instruction
2) alloc and store instruction
3) condition check and branch instruction
4) %1 = load i32, ptr %i
4) checks for validity of p
4) %2 = load i32, ptr %p
4) %3 = sub i32 %1, %2
4) store i32 %3, ptr %i
```

figure 3.

since our algorithm skips all instruction whose line number is same as faulty instruction, here this instruction is **%2 = load i32, ptr %p** whose line number is 4 and hence subtract instruction **%3 = sub i32 %1, %2** will be skipped so during runtime decrement will be skipped and control will enter in infinite loop.

So our algorithm can generate a control flow graph such that the path skipping the faulty instruction can be cyclic as shown in **figure 4** the path **B1 -> B2 -> B4 -> B5 -> B6 -> B1** is **cyclic**.

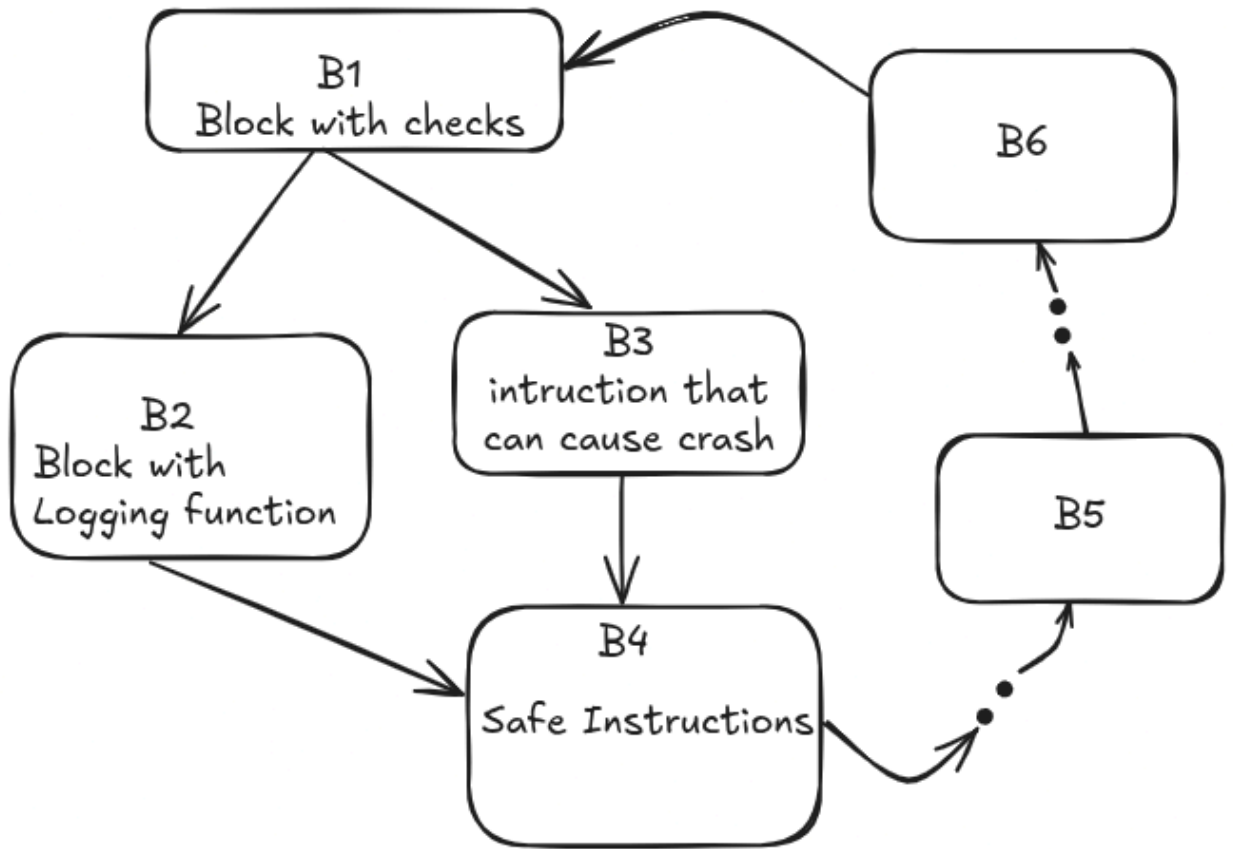


figure 4

To overcome that , we have implemented that if a faulty block is reachable by its successors or by bfs found safe block then we select our SafeExit block which is inserted in Control Flow Graph by our algorithm our next safeBlock. (it can avoid infinite loop but it can cause a surpass of lots of valid independent instruction execution and thus deadlock in multithreaded programs can occur since release of lock instruction can be skipped). So basically our algorithm is heuristic based.

Corresponding code :

<https://github.com/Shubham04567/Compiler-llvm/blob/main/outoftree/include/PassUtils.h>

under name of 1)findNextSafeInstruction 2)findNextSafeBlock

2. Dominance Fix :

We observe that after transformation of Control Flow Graph (CFG) (splitting and jumping to next safe instruction) , many such paths are generated which breaks the dominance of definition over use as shown in figure 5. Basically LLVM IR is static single assignment (SSA) based , it imposes the dominance rule that the path to use of a variable must pass through its definition.

Break of dominance rule causes LLVM error, pass run fails. To overcome this we come with our own Algorithm using bfs and Dominator Tree. A **Dominator Tree** is a tree data structure that represents the "dominance" relationships between the Basic Blocks in a Control Flow Graph (CFG).It basically has information of who dominates whom. In a Control Flow Graph, a block **A** is said to **dominate** block **B** if **every possible path** from the program's Entry Point to **B** must go through **A**. In this tree the edge from A to B tells that **A** dominates **B**. We are using these dominance information dominator trees (DT) have to fix the Dominance rule break issue.

So current algorithm is like that we take a instruction and for each of its operands (variables) we find the definition block (DefBB) and check the dominance of definition block over predecessors(PredBB) of Instruction's block because all path from entry to instruction's block come from its predecessors so if for any PredBB DefBB does not dominates that means there is a path from entry to Instruction's block which does not pass through definition block of instruction operand so Dominance rule breaks, like path **B1 -> B2 -> B3 -> use block** in figure 5. To fix that we are first storing such predecessors which are the cause of dominance violation (B3 in figure 5).

And then we are checking the reachability from DefBB block via its successors (downward BFS) , which are those blocks which are reachable from DefBB like B3 and use block of figure 5 and then We start Backward BFS (via predecessors) for each violating predecessors as source and find the first divergence (when we moving backward) in path from DefBB block to Instruction block , in figure 5 that first divergence is B2 -> B3 and redirect that control flow to safeExit to avoid Dominance Rule break. In this way the path from entry which comes to Instruction block without via DefBB block redirected to SafeExit block from its first divergence (this is critical because this is the point where the violating path merge to path from DefBB block to Instruction's block and that edge should be redirected to safeExit) and dominance fixed.

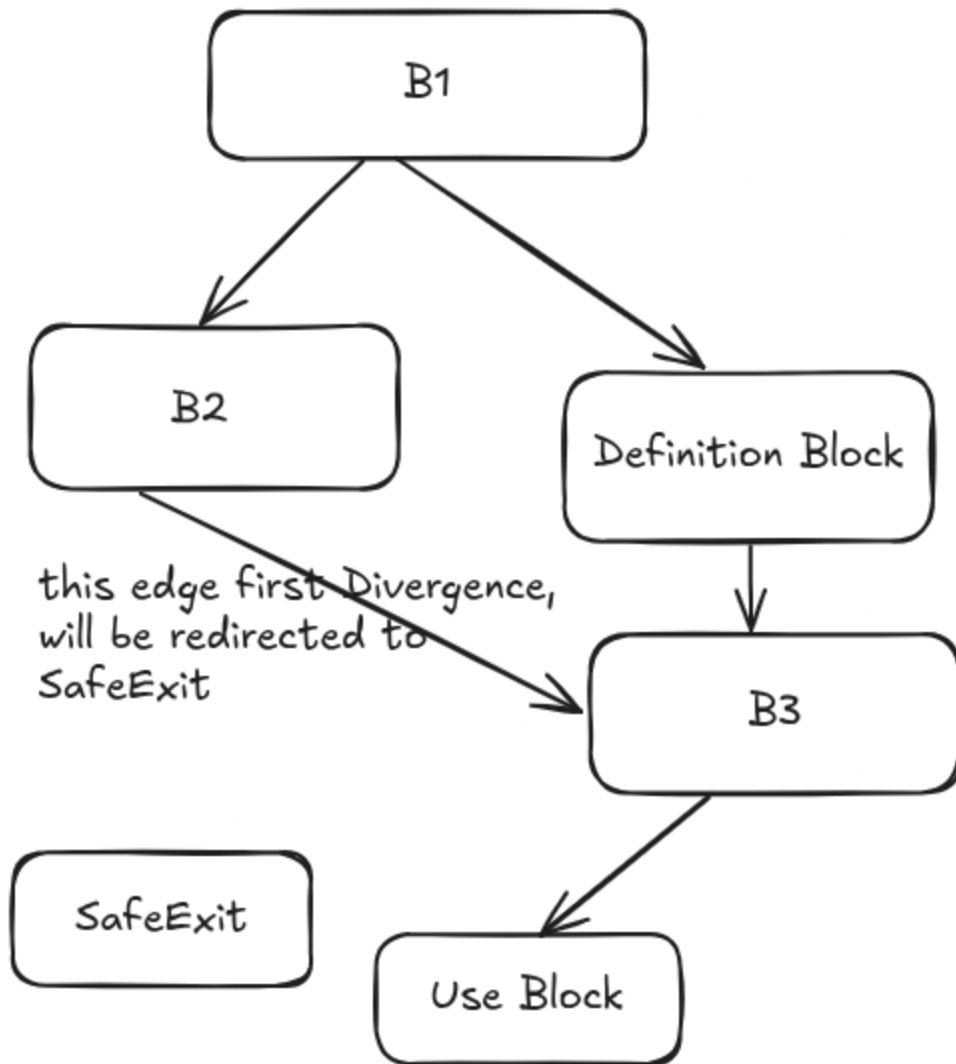


figure 5

Corresponding code :

<https://github.com/Shubham04567/Compiler-Illvm/blob/main/outoftree/include/PassUtils.h#L434> .

5.3 Pass Design and Implementation

5.3.1 AsanPass:

When a source file is instrumented by ASan (-fsanitize=address) it has many checks which jump to a block if found illegal instruction. That block have two instruction : 1) call to `__asan_report_*` reporting and aborting function and 2) unreachable terminator. This pass goal is to transform this aborting block to logging and redirecting block, Thus achieving our main goal of preventing program from runtime crash.

Whenever ASan inserts checks , the checking block has two successors : 1)Abort Block and 2) block containing instructions that need to be executed if checks pass. We use this Uniformity in our Pass.Pass iterate per function per block and identify the block which contain `__asan_abort_*` call and then start transformation of that block , first it create log message by using `getDebugLoc().getLine()` and function name and then use our function with signature `void __asan_log_violation(const char* msg, const char* srcfile)` to log this message. This log function by using LLVM api (such as `getOrInsertFunction`, `CreateCall`) and `IRBuilder` replace `__asan_report_*` function.Now we get a successor of the check block which has normal instruction and then by using the `NextSafeBlock` strategy, control flow redirection happens and unreachable is replaced by unconditional branch instruction jump to `safeBlock`.

Since block splitting and control flow redirection happen, there might be cases where SSA dominance requirements for operands of some instructions get violated. So after the above mentioned module level transformation , Our Dominance Fix strategy runs per function , per block and per instruction of source file , to check if there are any dominance violations and if any then fix them.

5.3.2 GEP Pass:

The GEP stands for `getelementptr`. GEP (`getelementptr`) is an LLVM instruction used to compute the address of a specific field or element inside aggregate data types (arrays, structs, nested aggregates).Using a base pointer and a sequence of indices, GEP calculates the effective address of elements inside structs (including nested fields), array elements (including multi-dimensional arrays), or combinations of both. Importantly, GEP does not perform memory access; it only performs address computation.

Since it does not perform any load and store instruction we decided to not put any validity check for base pointer, Asan also does not put. But in a test case it present in our repo : outoftree/test/miniGEPandMemcpyTest/testNULL.c where the base pointer is null , after run of AsanPass which we found that transformed IR is correct , control flow logic is also according to prevention of crash but still it crashes during runtime. When we check we observe that after asan checks instead of jumping to `asan_log_violation` block (before asan pass it was Asan abort block) it jumps to faulty block and execute faulty instruction. But as we put checks on the base pointer of GEP and redirect the control to `asan_log_violation` block , it passes (not crash at runtime) and similarly for more hard test cases.

This pass checks the validity of the base pointer of the GEP instruction. To do this, we use our custom function `int __my_is_valid_base_ptr(void*)`, which internally uses the `__asan_address_is_poisoned(void*)` and `__sanitizer_get_ownership(void*)` APIs provided by ASan to verify the base pointer. This check is inserted immediately above the GEP instruction.

The pass uses the `NextSafeBlock` strategy to identify a `SafeBlock`. In many cases, a corresponding ASan report block already exists after the GEP instruction. The pass attempts to find this next ASan report block (which becomes an ASan log block after the `AsanPass` runs). If it is not found,. Then, using the LLVM API `BasicBlock::Create`, it adds a `GEPAsanBlock` to the CFG in which `__asan_log_violation` function and unconditional jump to safe block is inserted.

Now we do block splitting at gep instruction so that it goes in a downward block and check by function in an upward block, then we perform rerouting of control flow. If the check results invalid base pointer it will go to asan log block, otherwise gep block (control continue to execute gep instruction).

Here also splitting , adding of blocks and rerouting of flow happen. Dominance fix run after compilation of module level transformation to fix dominance break.

5.3.3 Memcpymv Pass:

We observe the use of this pass when we see that in IR memcpy instruction is generated corresponding to dereferencing a struct pointer and storing another struct. Since that struct pointer can be null , freed so we need this pass. So source or destination or both if any pointer is invalid this pass will make it to log that and jump to the next safeblock.

We use almost similar strategies here as gep pass. Do the splitting at current (memcpy or memmove containing) instruction so that this instruction goes downward and checks inserted in the upward block. To insert this check the same function `__my_is_valid_base_ptr` is used. By using `NextSafeBlock` Strategy it finds the next safe block. Now Pass insert a new log block having `asan_log_violation` (`MemcpymvAsanBlock`) function and unconditional jump to

safeBlock. Now the original block which have checks change its terminator , if checks results atleast one of them is invalid it jumps to MemcpymvAsanBlock otherwise normal execution (execute memcpy or memmove instruction).

After this pass also we thought to run dominance fix , since adding blocks, splitting and rerouting can cause dominance violation error.

5.3.4 FreePass:

This pass Handles the almost all issues raise with free call : use after free, double free, freeing the allocated region by pointer which point at offset from the base address of the allocated region, double free or use after free by aliased pointer.

To achieve this check we use our custom function `int __my_is_valid_base_ptr(void*)` which internally uses the `__asan_address_is_poisoned(void*)` and `__sanitizer_get_ownership(void*)`, `void* __sanitizer_get_allocated_begin(void*)` APIs provided by ASan to verify the pointer used to call free.

The strategy is similar to Memcpymv pass. By Using NextSafeBlock strategy we found the next safe block and then splitting the current block at free instruction so that free comes in downward block and check are inserted inserted in upward block, then a new log block having `__asan_log_violation` function and unconditional jump to next safe block is inserted in CFG and now terminator of original block containing check function got changed, if check result invalid free pointer then jump to log block other wise jump to normal continued block containing free instruction.

Since dominance can break here also, we run dominance checks here also.

5.3.5 InitPtrPass:

This pass is required because we found that during memcpy or memmove or free execution or dereferencing , if the pointers are uninitialized, our functions `__my_is_valid_base_ptr` and `__my_is_valid_free_ptr` were not detecting them as illegal.

So, we decided to write a pass that runs first to find all uninitialized pointers and initialize them.

The strategy is simple: go into each function of the source file (module). First, store all the pointers that are allocated. Then in a second loop, store separately all the pointers for which a store instruction is performed (initialized). Now in the final loop, check for each pointer if it is present in the initialized pointer container. If not, insert a store null instruction for that pointer using IRBuilder right below the allocation instruction.

Since no control flow is changed, there is no need to run the Dominance check.

5.3.6. Realloc Pass

5.3.6.1. Introduction

Normally, if a program tries to read or write beyond the end of a block of memory it allocated (an **Out-of-Bounds or OOB access**), security tools like AddressSanitizer (ASan) detect the error and stop the program immediately.

Our system, **ASan-Recover**, tries to **detect** the OOB access on heap memory. Automatically enlarge the memory block using the `realloc` function with proper logging. Allow the program to continue execution successfully with the now-valid memory access.

5.3.6.2. Allocation and Base Pointer Tracking

To enable run-time memory management, all standard library calls to `malloc` are replaced with calls to the custom run-time function `tracked_malloc`. Crucially, the function ***findHeapBase*** implements a data-flow analysis to recursively trace back any memory access pointer (derived from *GetElementPtrInst*, *BitCastInst*, *LoadInst*, or *PHINode*) to its originating heap allocation instruction (`malloc`, `calloc`, etc.). This original base pointer is mandatory for the recovery mechanism (`realloc` must be called on the base address).

5.3.6.3. Access Instrumentation and Control Flow Splitting

For every *LoadInst* and *StoreInst* whose address operand traces back to a heap allocation, the pass instruments the instruction by splitting the containing basic block into a verification path and an error path. The original instruction is preceded by a call to the run-time function `isAddressValid(pointer,size)`.

Condition=`isAddressValid(Pointer,Size)`

Based on this condition, a conditional branch is inserted:

- If Condition is true, control proceeds to the okBB(safe Basic Block), which executes the original memory access.
- If Condition is false, control transfers to the errBB(error Basic Block), initiating the recovery process.

In the error path (errBB), the function `my_handle_illegal_access` is invoked, which attempts the memory resize. The resulting pointer, `new_pointer`, is then merged with the original pointer via a *PHINode* in the continuation block (contBB), ensuring the memory access proceeds with a valid, potentially relocated, address.

5.3.6.4. Pointer Variable Relocation

The most critical management happens when there is a derived pointer. If the memory access happen through local pointer at base or at any offset (e.g., accessed via a LoadInst on a stack-allocated pointer variable), the pass includes logic to:

1. Calculate the offset δ between the current (Out Of Bound) pointer and the original base pointer: $\delta = \text{calculate_pointer_offset}(\text{derived pointer}, \text{base_pointer})$.
2. Retrieve the newly allocated pointer for base pointer by realloc managed by run time structure (new_base_pointer).
3. Calculate the corrected pointer value for the variable:
 $\text{Pointervar_new} = \text{apply_offset_to_base}(\text{new_base_pointer}, \delta)$.
4. Update the original pointer variable on the stack with Pointervar_new. This ensures all future accesses using that variable target the correct memory region.

5.3.6.5. Dynamic Recovery via Runtime Library

The runtime library maintains two critical maps: alloc_map (base pointer to current size) and ptr_relocations (old base pointer to new base pointer after realloc).

The function **my_handle_illegal_access** drives the recovery:

1. It calculates the **needed size** as the sum of the current pointer's offset from the base and the accessed size: $\text{size_needed} = \text{Offset} + \text{Size}$.
2. It then allocates new memory by using $\text{realloc}(\text{base_ptr}, \text{needed})$.
3. Upon successful realloc, it updates alloc_map with the new base and size, and records the relocation in ptr_relocations.
4. It returns the newly calculated valid access address $\text{new_base_ptr} = \text{base_ptr} + \text{offset}$.

6. Testing:

We have tried to cover almost all types of errors that can be introduced in code that should be handled by our passes. The AsanPass test is simple: whenever there is illegal memory access (e.g out of bound array access or freed memory access) there should be logging in the same directory of source file with the same name and continued execution. It passes on all our current tests. For GEP pass we tried to have struct pointer which is null and then access its one of field , base pointer is freed and then after doing pointer arithmetic try to access and many more such test cases available in our directory are passed (means error line is logged and program continued for normal execution) , for memcpy and memmove Pass either destination is null or source is null or both are null or uninitialised all such and harder available test are passed. For freePass we tried use after free, double free , freeing the allocated location by pointer which is pointing to address inside the region, freeing the already freed pointer by its alias, by nested function and recursive function trying to simulate double free all such cases and even harder than this passed means logged and continued. For Realloc Pass we have tested across multiple

out of bound access for dynamically allocated memory , on each OOB access it allocates new memory for required size and updating base pointer. We tested with derived pointer also for that it first calculated Offset after base pointer then updated memory of new base pointer with an calculated offset. We tried with struct also it worked but if there is an intermediate pointer which is not being used during base pointer update but later access when it accessed it getting an older base pointer ,it will be our next goal to complete it using a comprehensive Data Flow Analysis.

We also tried multithreaded program and multisource files program , in multithreaded program due to our strategy inside loop if any error occurs , program jump to safeExit to avoid potential infinite loop, it is good for single threaded program , but bad for multithreaded program, jumping without taking care of releasing lock instruction is unsafe for multithreaded program, in case of erroneous loop (in case of our test/bfs/multithreaded.c) due to long jump without unlocking the lock cause program in deadlock. For multisource file program (in directory test/bfs) we have two version of bfs implemented with help of queue and adjacency list formed by linked list one is correct and one is faulty (before traversal some node is freed and freed allocation tried to access), we tested correct one properly executed and incorrect one also executed without crash and done proper logging.

We started with very basic version of safeblock selection strategy, detecting invalid pointer , but as test cases increases and become harder and harder we derived more and more resilient strategy. Our Dominance fix algorithm or checking pointer validity and jumping on safeexit if loop detected all are derived and enhanced by these test cases.

As we were increasing the number of test cases our passes were improving, by covering a very vast set of test cases we are confident that these passes (especially first 5 passes) can definitely work well. But still we are not very confident because with stress test cases , many bugs can come out.

All test case for first 5 pass are available at :

<https://github.com/Shubham04567/Compiler-Illvm/tree/main/outoftree/test>

7. Benchmarking

We have a script `install.sh` available at our directory `outoftree/install.sh` to make our tool '**usacclang**' system wide available. '**usacclang**' is a custom compiler tool built on Clang/LLVM based. Every source file compiled with `usacclang` will be ASan instrumented and all first five passes will run on that in order : **InitPtrPass** -> **AsanPass** -> **GEPPass** -> **MemcpymvPass** -> **FreePass**. Due to system wide availability like normal gcc and clang we can use it for benchmarking and compiling programs.

In our directory `outoftree/benchmarking` three multisource program for benchmarking is available under folder:

- 1) `threading` : having a program of multiple threads allocating memory and writing pointer to `shared_buffer` array and freeing the allocated location pointed by pointer at a randomly generated index of shared buffer if the pointer is not null and then goes to sleep for a random time period.
- 2) `bfs` : linked list is used to make queue and adjacency list and randomly graph generated and bfs is performed as node zero as source.
- 3) `replacement_policies` : different page replacement policy implementation fifo,LRU,approx LRU,random. And then the graph is plotted against a different set of pages : without locality, 80-20 workloads, sequential pages in loop.

We run all these three program by compiling using gcc, clang and our `usacclang` and we get :

For `threading`:

Compiler	Compile Time (s)	Run Time (s)
gcc	.442510400	.053739849
clang	.560530886	.050784701
usacclang	5.019396894	.155941832

Compile time ~10x , Runtime ~3x of our tool with respect to the other two.

For Bfs:

Compiler	Compile Time (s)	Run Time (s)
gcc	.482452592	3.480761425
clang	.447838279	2.554661953
usacclang	5.445373061	5.242540285

Compile time ~10x , Runtime ~2x of our tool with respect to the other two.

For replacement_policies:

Compiler	Compile Time (s)	Run Time (s)
gcc	.599617752	9.192926768
clang	.664302604	9.010801309
usacclang	7.442121512	42.123112113

Compile time ~10x , Runtime ~5x of our tool with respect to the other two.

Since gcc and clang have not used Asan and we know Asan makes programs 2x slower so they are already 2 times faster.

By comparison we can say that compile time of usacclang is almost 10 times slower and runtime depends on program types. If it involves a lot of memory operation there will be more memory checks and it becomes slower and more slower if those checks are in loop for example in case of replacement_policies it is 5 times slower than other compilers. For compile time our tool is slow because of finding the next safe block in almost all passes using bfs and dominance rule fix after each pass using bfs.

References:

1. <https://llvm.org/>
2. <https://www.cs.cornell.edu/~asampson/blog/llvm.html>
3. <https://llvm.org/pubs/2003-09-30-LifelongOptimizationTR.pdf>
4. https://simplifcpp.org/books/Designing_and_Developing_Compilers_Using_LLVM.pdf
5. https://drive.google.com/file/d/1qggqYMbKCSDOd_nXkBDAaAedGtdHbWZUS/view
6. <https://developer.android.com/ndk/guides/asan>
7. <https://llvm.org/devmtg/2014-04/PDFs/Talks/Passes.pdf>
8. <https://blog.trailofbits.com/2024/05/16/understanding-addresssanitizer-better-memory-safety-for-your-code/>